

Statistiche e Perché comprimere

Abbiamo finora visto le principali tecniche di index construction, anche nel mondo distribuito. Ci chiediamo però adesso: come possiamo risparmiare ancora di più in termini di spazio?

Tramite la **Compressione**. La compressione permette di:

- Usare meno spazio su disco (quindi risparmiare spazio)
- Tenere più dati in RAM e velocizzare il trasferimento dal disco alla memoria (quindi velocizzare il sistema)

Nel caso degli inverted index, la compressione riguarda principalmente il **Dizionario** (comprimerlo permette di renderlo abbastanza piccolo da stare in RAM, e tenere il dizionario in RAM è fondamentale per velocizzare le query per raggiungere rapidamente la posting list) e le **Posting List** (comprimerle significa ridurre lo spazio necessario su disco, il tempo per leggerle da disco, permette di mantenerne una parte più ampia in RAM). Infatti i grandi motori di ricerca tengono già una parte significativa delle posting in memoria, e ciò è possibile grazie alla compressione.

Si distinguono due tipi di compressione:

- **Lossless Compression:** la compressione è reversibile, e permette di recuperare esattamente i dati originali. E' quella che ci interessa per il nostro caso, perché non vogliamo perdere informazioni.
- **Lossy Compression:** la compressione non è reversibile, e non permette di recuperare esattamente i dati originali. Viene usata in contesti come immagini, audio, video, dove una perdita di qualità e aggiunta di rumore è accettabile per ottenere una maggiore compressione.

Prima di vedere i primi schemi di compressione, vediamo alcune statistiche relative al **Preprocessing** (che in un certo senso **rappresenta una compressione lossy** dal momento che riduce il numero di termini), Facendo sempre riferimento al RCV1:

size of	word types (terms)			non-positional postings			positional postings		
	dictionary			non-positional index			positional index		
	Size (K)	$\Delta\%$	cumul %	Size (K)	$\Delta\%$	cumul %	Size (K)	$\Delta\%$	cumul %
Unfiltered	484			109,971			197,879		
No numbers	474	-2	-2	100,680	-8	-8	179,158	-9	-9
Case folding	392	-17	-19	96,969	-3	-12	179,158	0	-9
30 stopwords	391	-0	-19	83,390	-14	-24	121,858	-31	-38
150 stopwords	391	-0	-19	67,002	-30	-39	94,517	-47	-52
stemming	322	-17	-33	63,812	-4	-42	94,517	0	-52

Unfiltered rappresenta l'index creato usando tutti i termini. Si osserva come il non-positional index e soprattutto il positional index rappresentino la principale fonte di spazio occupato.

- **No numbers:** se rimuoviamo i termini numerici, otteniamo una riduzione poco significativa (-2% nel dizionario e circa -8% nelle posting list). Si noti come la riduzione tra non-positional e positional sia circa la stessa: ciò è dovuto al fatto che in un documento (articolo di giornale nel caso di RCV1) un termine numerico appare pochissime volte, quindi per ogni posting == DocID tipicamente c'è anche solo una posting relativa alla posizione nel documento.
- **Case folding:** se applichiamo il case folding (trasformare tutte le parole in minuscolo) la riduzione del dizionario delle posting list è minima (perché i termini che differiscono solo per il case tendono ad apparire negli stessi documenti, quindi non si riducono le posting list). In particolare per le posting posizionali la riduzione è NULLA, in quanto ogni anche se i termini Apple e apple sono ora lo stesso, appaiono comunque ovviamente sempre nelle stesse posizioni nei documenti.
- **30 e 150 stopwords:** si ottiene una riduzione molto significativa nelle posting lists, in quanto le stopwords appaiono in moltissimi documenti e in moltissime posizioni. La riduzione del dizionario è invece minima, in quanto le stopwords sono solo una piccola parte del dizionario.
- **Stemming:** si portano i termini alla loro radice. La riduzione nel dizionario è molto significativa (si evitano duplicati come "run", "runs", "running"), ma la riduzione nelle posting list è minima (perché, soprattutto in articoli di giornale, i termini che differiscono solo per il suffisso tendono ad apparire negli stessi documenti, quindi non si riducono le posting list). In particolare per le posting posizionali la riduzione è NULLA, in quanto anche se i termini run, runs e running sono ora lo stesso, appaiono comunque ovviamente sempre nelle stesse posizioni nei documenti.

NB: differenza Stemming e Lemmatization: lo stemming è una tecnica più semplice che porta i termini alla loro radice, ma non garantisce che la radice sia una parola valida (ad esempio "amava", "amato" e "amare" diventano "am"). La lemmatization invece porta i termini alla loro forma base o lemma, che è una parola valida (ad esempio "running" diventa "run", ma "better" diventa "good"). La lemmatization è più precisa ma anche più complessa da implementare rispetto allo stemming.

Heap's Law

Abbiamo visto con **SPIMI** e **BSBI** che il costo di merge tra due dizionari ordinati è lineare nel numero di termini distinti M . Ma quanto può crescere M al crescere della collezione?

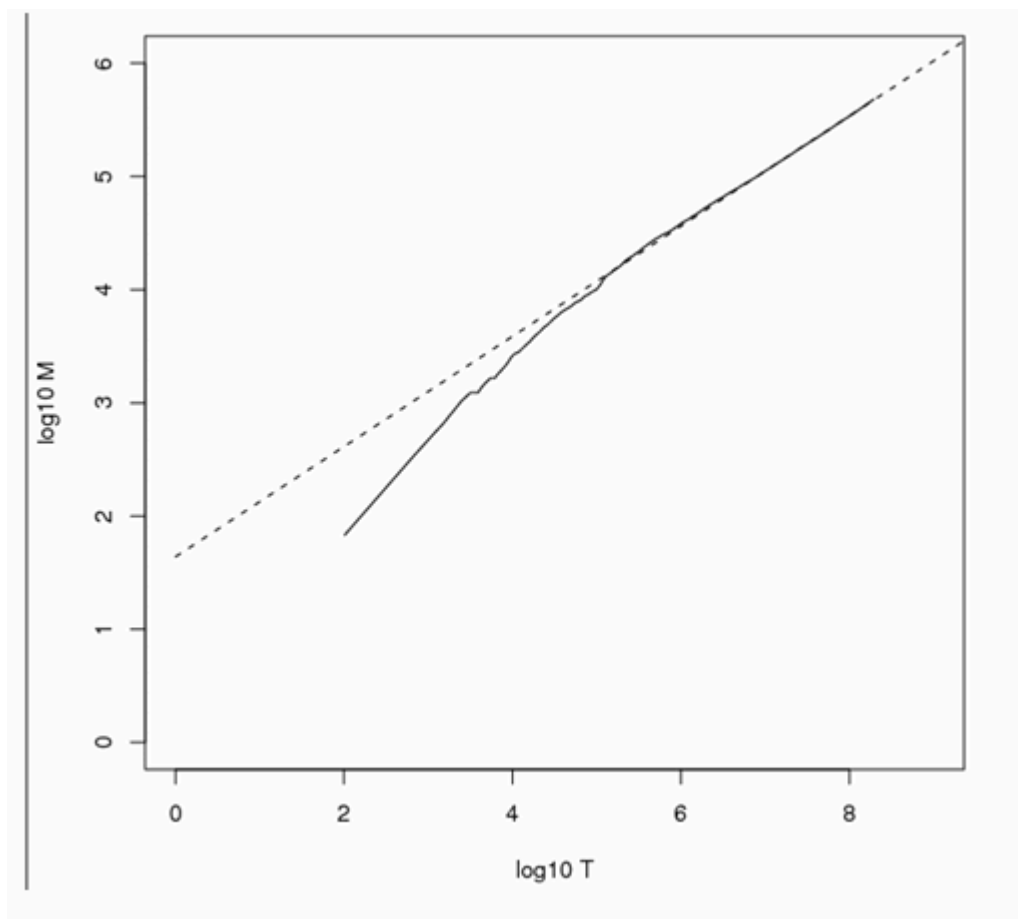
Non è possibile assumere un upper bound, in quanto anche limitandoci a 20 caratteri, il numero di stringhe possibili è (prendendo alfabeto inglese con 70 caratteri possibili) $70^{20} = 10^{37}$, numero intrattabile.

Si può "limitare" il numero di M al crescere della collezione sfruttando la legge *empirica* **Heap's Law**, sia M il numero totale di termini distinti del dizionario (terms) e T il numero totale di documenti nella collezione (documents), allora:

$$M = k \cdot T^b$$

dove k, b sono costanti empiriche che dipendono dalla lingua e dal dominio. Tipicamente $30 \leq k \leq 100$ e $b \approx 0.5$. In pratica, se cresce la collezione di documenti (T aumenta), cresce anche il numero di termini distinti M , ma non linearmente (b è 0.5, M cresce come la radice di T , il tutto per una costante). ES: se b è 0.5, allora se mettiamo 16 volte più documenti, abbiamo solo 4 volte più termini distinti.

Facendo il grafico log log della legge si ottiene una retta con coefficiente angolare b e intercetta $\log(k)$: $\log(M) = \log(k) + b \cdot \log(T)$. Si è verificato empiricamente che per RCV1 $b = 0.49$ e k è circa 44, e si vede infatti dalla figura che la legge è rispettata (es. per i primi 1kk token la legge predice 38.323 termini distinti, in realtà sono 38.365, differenza piccolissima).



Esercizi

- **Ex. 1:** Cosa succede se si lasciano come termini distinti gli errori ortografici presenti nella collezione alla Heap's Law? Includendo gli errori ortografici il numero di termini distinti M aumenta al crescere della collezione, quindi la curva di Heap's si alza.
- **Ex. 2:** 3000 termini distinti nei primi 10k tokens, 30k termini distinti nei primi 1kk tokens. Assumiamo che il motore di ricerca indicizzi 20.000.000.000 pagine ($2^{10^{10}}$), ognuna con 200 tokens in avg. Calcola M secondo la Heap's Law. Si tratta sostanzialmente di risolvere un sistema. Sappiamo Heaps $M = k \cdot T^b$, quindi per i primi 10k tokens abbiamo $3000 = k \cdot (10^4)^b$ e per i primi 1kk tokens abbiamo $30000 = k \cdot (10^6)^b$. Dividendo la seconda equazione per la prima otteniamo $10 = 10^{2b}$, da cui $2b = 1$ e quindi $b = 0.5$. Sostituendo b nella prima equazione otteniamo $3000 = k \cdot (10^4)^{0.5}$, da cui $k = 30$. Quindi $M = 30 \cdot T^{0.5}$. Per trovare il numero totale T di token, so che ho 20 miliardi di pagine ognuna con 200 token avg, quindi $T = 2 \cdot 10^{10} \cdot 200 = 2 \cdot 10^{10} \cdot 2 \cdot 10^2 = 4 \cdot 10^{12}$. Applico quindi Heaps:

IMMAGINE

Zipf's Law

La legge di Heaps limite empiricamente la dimensione di M al crescere di T . Ci chiediamo adesso se si possa fare un ragionamento simile per la frequenza dei termini.

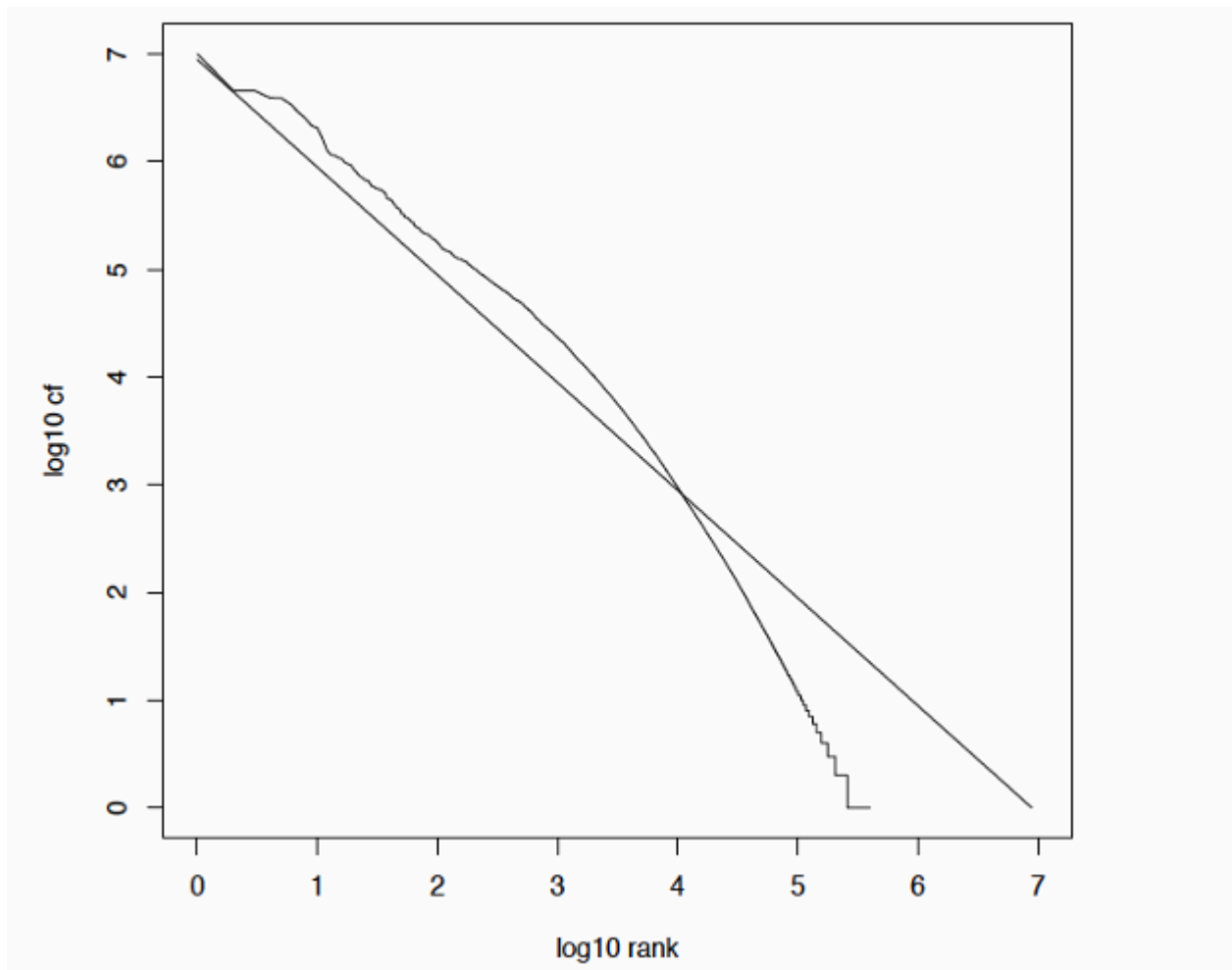
Nel linguaggio naturale, ci sono pochissime parole molto frequenti (es. the, of...), ma tantissime parole rare (termini tecnici, nomi propri...). Su questa osservazione si basa la **Zipf's Law**. Ordiniamo tutti i termini per frequenza decrescente (rango 1 il termine più frequente, rango 2 il secondo più frequente...). Sia cf_i la frequenza del termine di rango i in tutta la collezione, allora:

$$cf_i \frac{1}{i} = \frac{k}{i}$$

dove k è una costante di normalizzazione.

In pratica, la **Zipf's Law** afferma che la frequenza di un termine è inversamente proporzionale al suo rango, per cui il secondo termine più frequente appare circa la metà delle volte del termine più frequente, il terzo termine più frequente appare circa un terzo delle volte del termine più frequente, e così via. Ciò significa che **la frequenza cala molto rapidamente all'aumentare del rango**.

Come nel caso di Heaps, anche la Zipf's Law è una legge empirica e possiamo tracciare il grafico loglog da cui si ottiene $\log(cf_i) = \log(k) - \log(i)$, nuovamente una retta, stavolta con pendenza -1 (infatti al crescere di i , cf_i diminuisce). Come si osserva dal grafico empiricamente RCV1 rispetto abbastanza bene la Zipf's Law, anche se vi sono deviazioni dovuti al fatto che si tratta di una legge empirica. In particolare, nella coda finale la curva scende molto rapidamente perché nella collezione ci sono tantissimi termini rari.



Una conseguenza della legge è che se ricerchiamo una parola a **bassa frequenza**, questa ci consente di avere un sottoinsieme di documenti molto più piccolo e mirato.

Perciò, possiamo costruire un indice che da priorità alle parole con più bassa frequenza, e meno priorità a quelli con frequenza alta.

Compressione

Si studia la compressione per dizionario e postings, considerando l'inverted index classico visto finora (per query booleane) e senza considerare i positional indexes. Le idee che vediamo sono comunque estensibili. Le tecniche di compressione che vediamo sono tutte **lossless**, in quanto non vogliamo perdere informazioni.

Compressione del Dizionario

Quando arriva una query, la prima cosa da fare è trovare i termini nel dizionario e recuperare il puntatore alla posting list. Quindi è fondamentale che il dizionario sia in RAM, e per questo è importante comprimerlo. Anche se non si dovesse riuscire a tenere tutto il dizionario in RAM, è comunque importante comprimerlo di modo che si carichi più velocemente e si leggano meno dati da disco.

Naive Compression, Fixed Width

Si usa un array di lunghezza fissa. Ogni record del dizionario contiene **termine**, la **frequenza del termine** (come visto in precedenza importante per fare ranking dei risultati (termini più rari sono più importanti per identificare un documento specifico), ottimizzare le query (es. se una query contiene un termine molto raro, è più efficiente partire da quello) e calcolare TF-IDF) e il **puntatore alla posting list**.

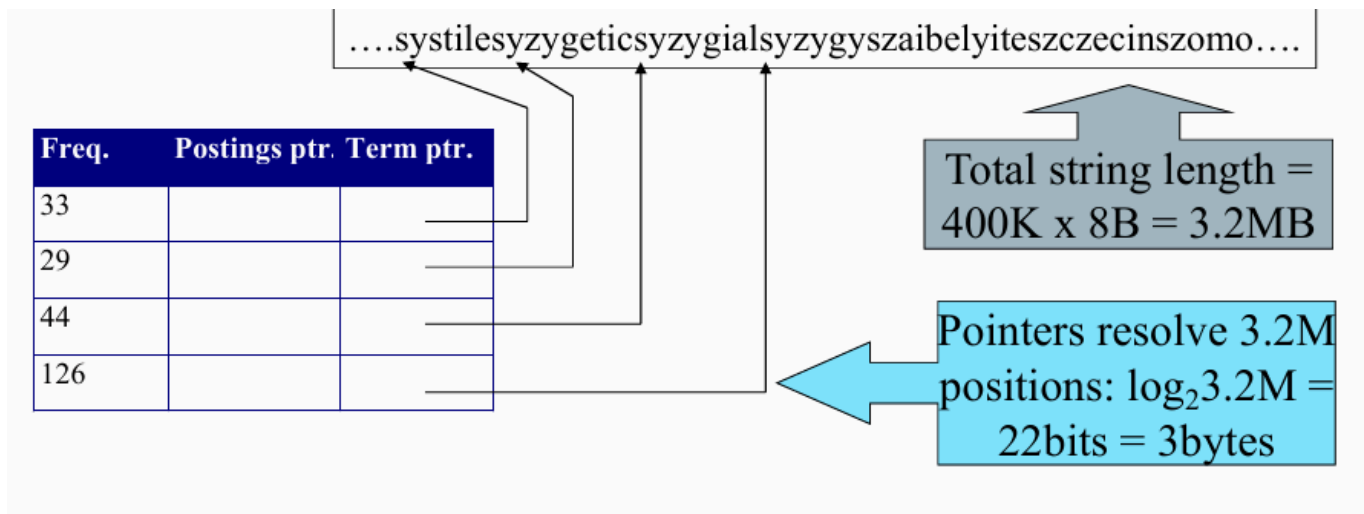
Si può ipotizzare di usare 20 byte per il termine, 4 byte per la frequenza e 4 byte per il puntatore, quindi 28 byte per ogni record del dizionario. Con 400k termini distinti --> $400k \cdot 28 \text{ byte} = 11.2kk \text{ byte} = 11.2 \text{ MB}$.

Tuttavia questa soluzione è molto inefficiente, in quanto il termine è di lunghezza variabile (es. "a" è un termine da 1 byte, ne occuperebbe comunque 20 con questa tecnica). Inoltre fissando il numero di byte potrebbero esserci parole particolarmente lunghe o complesse che non rientrano nei 20 byte, e quindi non possono essere rappresentate.

Perché, sapendo che nella lingua inglese in media un termine è lungo 5 caratteri, non ci si adatta di conseguenza? Perché la media è sui token, non sui term del dizionario (che spesso sono più lunghi per via dello stemming e preprocessing in generale). Facendo una media sui term per il dizionario nel caso di RCV1 si ottiene una lunghezza media di 8 caratteri, quindi 8 byte per il termine.

Dictionary as a String

Invece di memorizzare ogni termine come un record di lunghezza fissa, si può memorizzare l'intero dizionario come una stringa, con i termini concatenati uno dopo l'altro. Per capire dove inizia ogni termine, si può indicare la sua posizione iniziale nella stringa. Dove vengono salvati questi puntatori? Nella tabella del dizionario, che ora contiene per ogni termine solo **frequenza**, **puntatore alla posting list** e **puntatore alla posizione del termine nella stringa**. In questo modo si risparmia spazio, in quanto non si devono allocare 20 byte per ogni termine, ma solo lo spazio effettivamente necessario per memorizzare i termini.



Come detto prima, sapendo che in media un termine del dict occupa 8 byte, si ha una lunghezza totale della stringa di $400k \cdot 8 \text{ byte} = 3.2 \text{ MB}$. La tabella del dizionario invece ora contiene solo 4 byte per la frequenza, 4 byte per il puntatore alla posting list, mentre per il puntatore alla posizione del termine si possono usare 3 byte in quanto il **puntatore punta a un totale di 3.2 MB di dati** $\rightarrow \log(3.2 \cdot 10^6) = 22 \text{ bit}$, **quindi 3 byte sono sufficienti**. Quindi in totale $400k \cdot (4 + 4 + 3) \text{ byte} = 4.4 \text{ MB}$ e aggiungo i 3.2 MB della stringa, per un totale di totale di 7.6 MB, quasi la metà rispetto alla soluzione naive.

Blocking

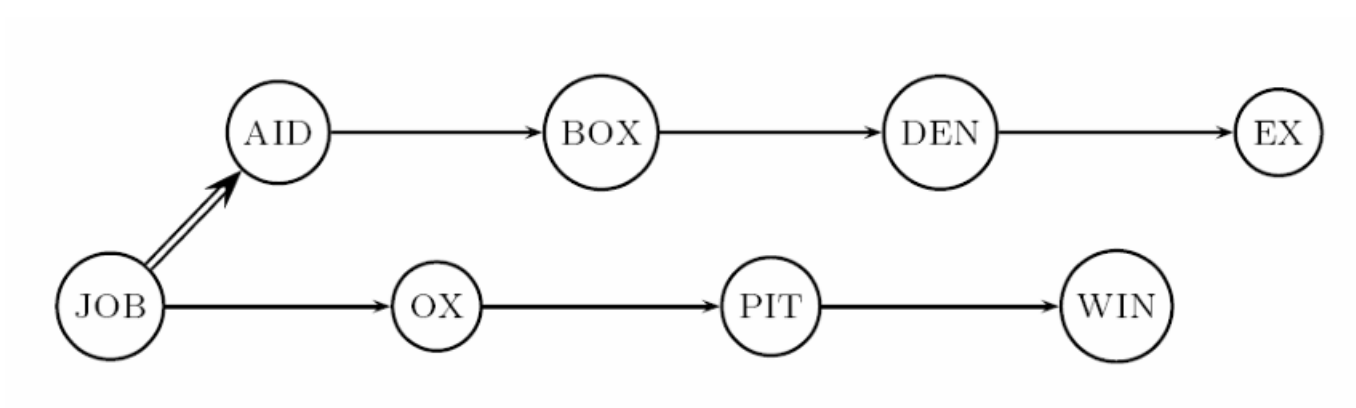
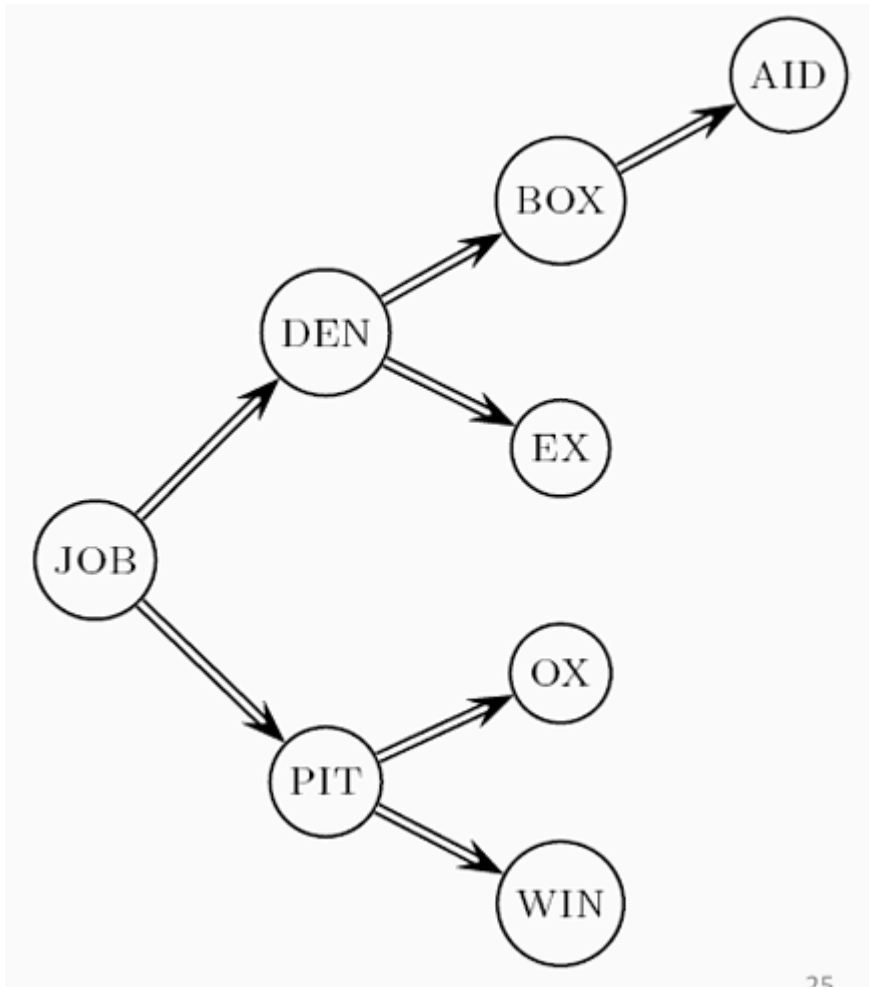
Si può ulteriormente comprimere il dizionario usando la tecnica del **blocking**. Con la stringa salvavamo per ogni termine anche il puntatore alla sua posizione. L'idea del blocking è invece di memorizzare il puntatore solo al primo termine di ogni blocco di k termini. In questo modo si evita di dover memorizzare un puntatore per ogni termine, ma solo per il primo termine di ogni blocco. **Ma come riconoscere i termini dentro ogni blocco?** Bisogna salvare anche la lunghezza di ciascun termine, e la si mette subito prima di ogni termine nella stringa. La stringa passa quindi all'essere qualcosa del tipo "7systile9syzygetic8syzygial6syzygy". Si risparmia in memoria dal momento che rappresentare le lunghezze richiede meno spazio rispetto a rappresentare i puntatori.

Ad esempio, usando un blocco da $k = 4$ termini nel caso RCV1, si ha un puntatore ogni 4 termini, e una lunghezza per termine (dove basta un byte per lunghezza) $\rightarrow$ per ogni blocco $3 + 4 = 7 \text{ byte}$. Senza blocking, per ogni termine avevamo 3 byte per il puntatore alla posizione, quindi $4 \cdot 3 = 12 \text{ byte ogni 4 termini}$. *In totale quindi 4 byte per frequenza, 4 byte per puntatore alla posting list, e 7 byte ogni 4 termini per blocco* $\rightarrow 100k \cdot 7 + 400k \cdot (4 + 4) \text{ byte} = 3.9 \text{ MB}$, e infine aggiungiamo i 3.4 MB della stringa, per un totale di 7.1 MB $\rightarrow$ risparmio 0.5 MB rispetto a prima.

Si risparmierebbe ancora di più usando k più grande, perché non sceglierlo? Se k troppo grande, risparmio si nei puntatori, ma **la ricerca nel dizionario per un termine diventa più**

lenta (infatti, una volta raggiunto il blocco con il puntatore, scansionare il blocco per trovare il termine richiede $O(k)$ operazioni).

Esempio con l'albero:



Ricordiamo che il binary search tree funziona come segue: data una parola nella query, per trovarla nel dizionario la confronto con ogni nodo partendo dalla radice. Se la parola è lessicograficamente minore del nodo, vado a sinistra, altrimenti vado a destra. Ipotizzando che ogni termine abbia la stessa probabilità di apparire in una query, **la ricerca senza blocking** richiede nel caso degli 8 term in figura in media 2.6 confronti. Con il blocking invece si ha u

confronto per trovare il blocco e poi di deve scorrere il blocco linearmente, nell'esempio in media 3 confronti.

Esercizi

- **Ex. 1:** Stima lo spazio necessario con blocking per $k = 4, 8, 16$. Per $k = 4$ già fatto sopra. Per una formula notevole: con blocking freq 4 byte 4, posting ptr 4 byte, term string 8 byte, lunghezza termine 1 byte, puntatore al blocco 1 ogni k termini, in media quindi $3/k$ byte per termine. Quindi il costo per termine è $4 + 4 + 8 + 1 + 3/k$ byte = $17 + 3/k$ byte. Per $k = 8 \rightarrow 17 + 3/8 = 17.375$ byte per termine, quindi $400k \ 17.375 \text{ byte} = 6.95 \text{ MB}$. Per $k = 16 \rightarrow 17 + 3/16 = 17.1875$ byte per termine, quindi $400k \ 17.1875 \text{ byte} = 6.87 \text{ MB}$. Si noti come il risparmio rispetto ai 7.6 MB della soluzione senza blocking sia ridotto man mano che k aumenta.
- **Ex. 2: DA FARE.** Stima base del numero di confronti $(k) \approx \log_2(kN) + k/2$ dove N numero di termini, $\log(N/k)$ confronti per trovare il blocco nell'albero e $k/2$ in media costo per trovare il termine nel blocco.

Front Coding

Idea: se i termini del dizionario sono in ordine lessicografico, è molto probabile che termini vicini condividano un prefisso comune (es. "automata", "automate", "automatic").

Invece di memorizzare ogni termine per intero, si può memorizzare solo il prefisso comune e poi la parte che cambia.

In questo modo si arriva a risparmiare ancora più spazio, secondo una stima su RCV1 usando blocking con $k = 4$ e front coding si arriva a 5.9 MB, contro gli 11,2 con il naive Fixed Width.

Compressione delle Posting List

Abbiamo risparmiato un pò di spazio comprimendo il dizionario, ma la parte più grande dell'inverted index è rappresentata dalle posting list, quindi è fondamentale comprimerle (tipicamente le posting list occupano almeno 10 volte spazio rispetto al dizionario, e non di rado anche 100 volte).

Nel caso dell'inverted index per query booleane, ogni posting rappresenta come sappiamo un docID. Usando interi standard, servirebbero quindi 4 byte = 32 bit per rappresentare ognuno. Tuttavia in teoria non ne servono così tanti, dipende dal numero di documenti nella collezione. Nel caso di RCV1, la collezione ha 800k documenti, e per distinguerli mi bastano $\log(800k) = 20$ bit, quindi 3 byte per rappresentare ogni docID.

Tuttavia ciò non basta, vogliamo risparmiare molto di più. Sappiamo infatti che alcuni termini appaiono in moltissimi documenti (es. stopwords), mentre altri appaiono in pochissimi documenti (es. termini tecnici). Salvare un termine come "arachnocentric" (che magari appare

in un solo documento o pochi più) usando 20 bit, va anche bene. Al contrario per un termine come "the", che appare in moltissimi documenti, usare 20 bit per ogni posting diventa insostenibile (sarebbe più conveniente usare una maschera booleana, dove ogni bit rappresenta un documento, e il bit è 1 se il termine appare in quel documento, altrimenti è 0).

Idea: i docID sono ordinati nelle posting list, quindi invece di memorizzarli direttamente possiamo memorizzare i gap: es. prima posting docID = 2, poi 3, 4, 7, 10 --> memorizzo 2, 1, 1, 3, 3. Quindi per "the" il problema si risolve, memorizzo gap che sono piccoli e quindi codificabili con pochissimi bit. Per i termini rari i gap sono grandi quindi non si risparmia molto, però non si peggiora rispetto ai docID assoluti.

Tuttavia per far sì che ciò funzioni è necessario un **Variable Length Encoding**: non voglio usare lo stesso numero di bit per ogni gap, ma adattarli in base al numero di bit necessari per rappresentarlo.

Variable Length Encodings : Unary and Gamma Codes

Vogliamo quindi un sistema per cui ogni docID possa avere lunghezza variabile. La prima idea naïve è usare lo **unary code**: per rappresentare un intero n , si usano n bit 1 seguiti da un bit 0. Ad esempio, per rappresentare il numero 5, si userebbero 5 bit 1 seguiti da un bit 0: "111110". Per rappresentare il numero 3, si userebbero 3 bit 1 seguiti da un bit 0: "1110". Quindi se abbiamo ad esempio il documento docID 13, questo viene salvato come 11111111111110.

Il problema dello unary è che per numeri grandi diventa lunghissimo. Sarebbe ottimale se la probabilità di avere un numero n decresce esponenzialmente ($p(n) = 2^{-n}$), in questo modo i numeri piccoli (gap piccoli) sono molto più probabili e quindi occupano pochi bit, ma ciò non è realistico in collezioni di documenti gigantesche.

Un'idea migliore è quella del **Gamma Code**, che sfrutta unary. L'idea è la seguente: dato un intero positivo G , il gamma code lo rappresenta come **length** e **offset**:

1. Scrivi G in binario, es. $13 = 1101$
2. Dato che ogni numero in binario inizia con 1, risparmiamo e non lo salviamo. Otteniamo quindi l'**offset**: 101
3. Calcoliamo la lunghezza dell'offset, che è 3, e la rappresentiamo in unary: 1110
4. Concatenando length e offset otteniamo il gamma code: 1110101

Questo codice è codificabile senza ambiguità: leggo gli 1 fino allo 0 identificando length, poi leggo length bit per identificare l'offset e aggiungo il primo 1 per ricostruire il docID originale.

number	length	offset	γ -code
0			none
1	0		0
2	10	0	10,0
3	10	1	10,1
4	110	00	110,00
9	1110	001	1110,001
13	1110	101	1110,101
24	11110	1000	11110,1000
511	111111110	11111111	111111110,11111111
1025	11111111110	0000000001	11111111110,0000000001

Quanto ci costa? Dato un numero G , per rappresentare l'offset circa $\log()$ bit e per rappresentarne la lunghezza sempre $\log()$ bit, quindi in totale circa $2\log()$ bit. Inoltre il gamma code può essere usato per qualsiasi distribuzione, è ottimale in pratica se la probabilità di avere un numero n è $1/(2n^2)$

Nonostante il gamma code sia molto elegante e sembri una soluzione ottimale, in realtà non è molto usato nei sistemi reali. Infatti **i gamma code lavorano a singolo bit**, le macchine per propria natura lavorano a blocchi di byte (parole macchina: 8, 16, 32 o 64 bit). Se la codifica mette un numero in una sequenza di bit che inizia in un punto qualsiasi e finisce a metà tra due parole macchina, allora per leggerlo bisogna fare più lavoro --> decodifica poco efficiente.

Per tale ragione, in pratica, nei sistemi reali, si utilizzano codifiche **byte-aligned** (o word-aligned), come vediamo con la tecnica **Variable Byte VB code**.

Variable Byte (VB) Code

Invece di lavorare bit per bit come i gamma code, si lavora a blocchi di byte. Dato un gap G , si vuole usare il numero minimo di byte per rappresentarlo. In particolare:

- Ogni byte dedica il primo bit a indicare se quello è l'ultimo byte del numero, mentre gli altri 7 bit vengono usati per rappresentarlo. Se il primo bit è 0, significa che ci sono ancora byte da leggere per completare il numero, se invece è 1, significa che quello è l'ultimo byte del numero.
- Quindi, se $G \leq 127$, basta un byte per rappresentarlo (perché 7 bit possono rappresentare numeri fino a 127), altrimenti si usano più byte, e solo l'ultimo byte avrà il primo bit a 1.

Es. se $G = 5$, in binario 101. Quindi dato che ovviamente entra in 7 bit si rappresenta come 100000101. Invece un numero come 824 in binario è 1100111000, spezzandolo si ottiene 0000110 1 0111000.

Nonostante VB sprechi spazio se ci sono gap molto piccoli, sono così tanto più efficienti da decodificare rispetto ai gamma code, che in pratica sono molto più usati nei sistemi reali.

Data structure	Size in MB
dictionary, fixed-width	11.2
dictionary, term pointers into string	7.6
with blocking, $k = 4$	7.1
with blocking & front coding	5.9
collection (text, xml markup etc)	3,600.0
collection (text)	960.0
Term-doc incidence matrix	40,000.0
postings, uncompressed (32-bit words)	400.0
postings, uncompressed (20 bits)	250.0
postings, variable byte encoded	116.0
postings, γ -encoded	101.0

Si osserva dalla tabella come con il gamma encoding si risparmiino 15 mega in RCV1. Non un danno eccessivo, il tradeoff per l'efficienza è molto conveniente. Importante puntualizzare come la Term-doc incidence matrix occupa tantissimo spazio (40GB) in quanto è una **rappresentazione densa**. Non appena infatti si passa a rappresentazioni **sparse** come l'inverted index delle postings, lo spazio si riduce esponenzialmente.

I sistemi attuali lavorano con VB ma con alcune ottimizzazioni, ad esempio per essere word-aligned (32-64 bit), oppure decodificando più gap alla volta, oppure ancora assumendo un maximum gap size (i.e. si progetta un formato in cui si dice "so rappresentare gap fino a una certa soglia, se arriva un gap più grande del previsto so un flag speciale per rappresentarlo". Conviene perché molti gap piccoli mentre gap grandi più rari).

Group Variable Integer Code e Simple-9

Riguardo l'idea di decodificare più gap alla volta, vediamo brevemente il **Group Variable Integer Code**. L'idea è di codificare 4 interi (gap) alla volta usando da 5 a 17 byte. Il primo byte contiene 4 campi da 2 bit, uno per ciascun intero, che ne indicano la lunghezza in byte. Dopodiché seguono i byte effettivi che codificano i 4 interi. Ogni intero può occupare 1, 2, 3 o 4 byte (per questo 2 byte per indicarne la lunghezza bastano).

Ad esempio, se i 4 gap da codificare sono 5, 824, 3 e 1000 allora il primo byte sarà 01 10 01 10 (perché 5 e 3 occupano 1 byte, mentre 824 e 1000 occupano 2 byte), seguito dai byte che rappresentano i gap.

Questa logica è ancora più spinta con **Simple-9**, dove si cerca di essere allineati con la parola macchina da 32 bit. Si prende il blocco da 32 bit e si cerca di codificare il maggior numero possibile di gap al suo interno. I 4 bit più significativi del blocco sono usati per dire che formato sto usando (i.e. quanti gap sto codificando e quanti byte occupano) mentre i restanti 28 bit sono usati per codificare i gap. Concretamente, i selector sono 9 (da qui il nome Simple-9):

- selector 0: codifico un numero da 28 bit
- selector 1: codifico 2 numeri da 14 bit
- selector 2: codifico 3 numeri da 9 bit

... e così via fino al selector 8 che indica che sto codificando 28 numeri da 1 bit. In questo modo si cerca di massimizzare il numero di gap codificati in un blocco da 32 bit, e quindi di minimizzare lo spazio occupato.

Il principio funziona molto bene dal momento che se ho gap molto piccolo (magari tutti 1) posso metterne addirittura 28 in una sola word.

Quindi Simple-9 simile a Group Variable Integer Code, ma invece di lavorare in byte prendendo 4 bit per indicare le lunghezze dei 4 gap successivi lavora con word da 32 bit e usa un selector unico.

Conclusioni

In conclusione, con queste tecniche di compressione, si può costruire un inverted index per query booleane molto efficiente in termini di spazio:

- L'indice può arrivare a occupare **solo il 4% della dimensione totale della collezione**, oppure circa il **10-15% della sola dimensione del testo della collezione**.

Si noti tuttavia che finora abbiamo ignorato il **positional index**, che come visto è la parte più grande dell'inverted index. Le tecniche per la compressione sono sempre le stesse, ed il risparmio resta comunque molto significativo.